16

BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

# ReentrantLock: Açık Kilitler, Adil (Fair) vs Haksız (Non-Fair) & Kesilebilirlik

ReentrantLock: Explicit Locking, Fair vs Non-Fair & Lock Interruptibility

## ⚡ Bellek & Eşzamanlılık Akış Şeması

## ⚡ Concurrency & Memory Schematic

```
SYNCHRONIZED vs REENTRANTLOCK MİMARİ KARŞILAŞTIRMASI

ÖZELLİK                 SYNCHRONIZED              REENTRANTLOCK
-------------------------------------------------------------------
Kilit Bırakma           Otomatik (Compiler)       Manuel (finally { unlock(); })
Zaman Aşımı (Timeout)   YOK (Sonsuza dek bekler)  VAR: tryLock(5, SECONDS)
Kesilebilirlik          YOK (Kesilemez)           VAR: lockInterruptibly()
Adillik (Fairness)      Sadece Haksız (Barging)   Fair / Non-Fair seçilebilir
Çoklu Kuyruk            Tek bir WaitSet           Birden fazla Condition nesnesi

NON-FAIR LOCK (Haksız Kilit - Yüksek Throughput):
Yeni gelen thread kuyrukta bekleyenleri beklemeden doğrudan kilide saldırır (Barging)!
Context switch maliyetinden kaçar, %400 daha hızlıdır!
```

## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Geleneksel anahtarlar (synchronized) basittir; kapıyı açar girersiniz, çıkınca kapı arkadan otomatik kilitlenir. Ama diyelim ki kapının önünde 3 saat beklemek istemiyorsunuz: 'Eğer 10 saniye içinde açılmazsa ben gidiyorum' demek istiyorsunuz (`tryLock(timeout)`). Ya da yangın çıktığında bekleyen adamı kolundan çekip dışarı çıkarmak istiyorsunuz (`lockInterruptibly()`). İşte ReentrantLock, cebinizde zamanlayıcısı, acil durum butonu ve sıra numarası olan son teknoloji akıllı kilit sistemidir!

### 🇬🇧 ENGLISH (ELI5)

An intrinsic lock (`synchronized`) is an old-fashioned deadbolt: you wait until it opens, with no escape. ReentrantLock is an enterprise smart lock: 1) You can say: 'If the door doesn't unlock in 5 seconds, I'm leaving' (`tryLock(timeout)`). 2) In a fire alarm, a thread waiting for the lock can be canceled immediately (`lockInterruptibly()`). 3) You can enforce a strict first-come-first-served line (Fairness) or allow energetic newcomers to barge in immediately for 4x higher throughput (Non-Fair).

## 🔍 MAANG Seviyesi Teknik Derinlik

### 🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ReentrantLock ve AQS Derinliği: 1) AbstractQueuedSynchronizer (AQS): ReentrantLock, temelinde çift yönlü bir FIFO bekleme kuyruğu (CLH lock varyantı) ve bir `volatile int state` tutar. 2) Fair vs Non-Fair Kilit: Non-fair kilit (varsayılan) 'Barging' uygular. Yeni gelen thread, kuyrukta bekleyenler olmasına rağmen önce bir CAS ile kilidi kapmaya çalışır. Eğer o an kilit boştaysa, kuyruk uyandırma (unpark) ve context-switch gecikmesini atlayarak kilidi alır; throughput dramatik yükselir. Fair lock ise daima kuyruk başındaki thread'e verir, adaleti sağlar ancak gecikmeyi katlar. 3) finally Bloğu Zorunluluğu: `lock.unlock()` daima `try-finally` bloğunun finally kısmında olmalıdır; aksi halde bir exception fırladığında kilit sonsuza dek kilitli kalır ve sistem felç olur!

## 🔍 MAANG-Level Technical Deep-Dive

### 🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Internals of ReentrantLock: 1) AQS Foundation: ReentrantLock wraps `Sync extends AbstractQueuedSynchronizer`. State 0 represents unlocked; state >= 1 denotes reentrant holds by the owning thread. 2) The Non-Fair Barging Advantage: `NonfairSync` attempts an immediate CAS on `state` before enqueuing. If an unparking thread has not yet acquired the CPU core, a newly arriving active thread sneaks in, eliminating two context-switch penalties. This yields massive throughput gains at the expense of potential thread starvation. 3) Non-Negotiable Idiom: Because the compiler cannot enforce automatic release, `lock.lock()` must immediately precede a `try` block, with `lock.unlock()` residing strictly inside `finally`.

## ⚡ Modern Java 25 LTS Kodu

## ⚡ Modern Java 25 Implementation

```java
// Java 25: ReentrantLock ile Zaman Aşımlı (tryLock) ve Kesilebilir Kilit
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

public class AdvancedLockingDemo {
    private final ReentrantLock lock = new ReentrantLock(false); // Non-fair (Hızlı)

    public void processPayment() {
        lock.lock();
        try {
            IO.println("Ödeme işlemi kilitlendi: " + Thread.currentThread().getName());
        } finally {
            lock.unlock(); // ⚠ KESİNLİKLE finally içinde olmalıdır!
        }
    }

    public boolean tryProcessWithTimeout() throws InterruptedException {
        // 2 saniye içinde kilit alamazsa vazgeç (Deadlock önleyici!)
        if (lock.tryLock(2, TimeUnit.SECONDS)) {
            try {
                IO.println("Zaman aşımlı kilit alındı!");
                return true;
            } finally {
                lock.unlock();
            }
        } else {
            IO.println("Kilit meşguldü, işlem timeout'a uğradı!");
            return false;
        }
    }
}
```

### ⚠️ Deadlock, Race Condition & Mülakat Tuzağı

#### 🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Ölümcül Kodlama Hatası: `lock.lock()` satırını `try` bloğunun İÇİNE yazmayın! Eğer `lock()` anında bir hata fırlatırsa, `finally` bloğu çalışır ve sahip olunmayan bir kilit için `unlock()` çağrılır (`IllegalMonitorStateException`). Kural: `lock.lock(); try { ... } finally { lock.unlock(); }`!

### ⚠️ Concurrency Hazard & Interview Trap

#### 🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Severe Anti-Pattern: Never place `lock.lock()` INSIDE the `try` block! If `lock()` throws before acquisition, `finally` executes `unlock()`, triggering an `IllegalMonitorStateException` that masks the root cause. Idiom: Invoke `lock.lock()` strictly on the line preceding `try`.

### 🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ReentrantLock; tryLock(), timeout ve interruptibility esnekliği sunar.
- Non-fair kilitler context switch maliyetini baypas ettiği için çok daha yüksek throughput üretir.
- unlock() çağrısı DAİMA finally bloğu içerisinde yer almalıdır.

### 🎯 Key Takeaways for the Interview

- ReentrantLock provides tryLock(), timeouts, and interruption capabilities.
- Non-fair locks bypass context switches via barging, delivering superior throughput.
- unlock() must strictly execute inside a finally block to prevent resource leaks.